

PROJECT REPORT

Problem Statement-1:

AI-Based Network Security Threat Detection
(SQL Injection, XSS, and Normal Traffic Classification)

AI-Based Network Security Threat Detection
(SQL Injection, XSS, and Normal Traffic Classification)

Team Details

Role	Name	Details	Contributions
Team Leader	S. Sai Varma	VU22CSEN0102367- Student, Department of Computer Science and Engineering, GITAM Deemed to be University, Visakhapatnam, 530045 Andhra Pradesh, India	Project architecture, model training (train.py), Streamlit UI (app.py), GitHub management
Team Member	S. Likith Reddy	VU22CSEN0101831- Student, Department of Computer Science and Engineering, GITAM Deemed to be University, Visakhapatnam, 530045 Andhra Pradesh, India	Data preprocessing (preprocess.py), feature extraction, model evaluation
Team Member	S. Prapul Sai	VU22CSEN0100204- Student, Department of Computer Science and Engineering, GITAM Deemed to be University, Visakhapatnam, 530045 Andhra Pradesh, India	Dataset preparation, Excel prediction tool, testing and debugging
Faculty Guide	Dr. Satyabrata Dash	Assistant Professor, Department of Computer Science and Engineering, GITAM Deemed to be University, Visakhapatnam, 530045 Andhra Pradesh, India. satyabrata.cse@gmail.com	Guidance on ML techniques and ensured timely project progress

AI-Based Network Security Threat Detection (SQL Injection, XSS, and Normal Traffic Classification)

Abstract

This project presents an AI-based approach for detecting and classifying common web security threats—namely SQL Injection (SQLi) and Cross-Site Scripting (XSS)—within network traffic. Developed under the Intel Unnati Industrial Training 2025, the system leverages Natural Language Processing (NLP) and Machine Learning (ML) to automate threat detection from log files and URLs. Using TF-IDF vectorization and a Logistic Regression model, the system efficiently classifies inputs into SQLi, XSS, or Normal traffic categories. The trained model is deployed via an interactive Streamlit interface, allowing real-time threat assessment. A well-structured dataset of over 156,000 entries was curated and preprocessed for optimal model training. The solution achieved high accuracy in both training and testing phases, supported by evaluation metrics like precision, recall, and confusion matrix. With robust architecture, clean UI, and automation, this system addresses the critical need for faster, more reliable cyber threat detection, offering a scalable tool for enhancing web application security.

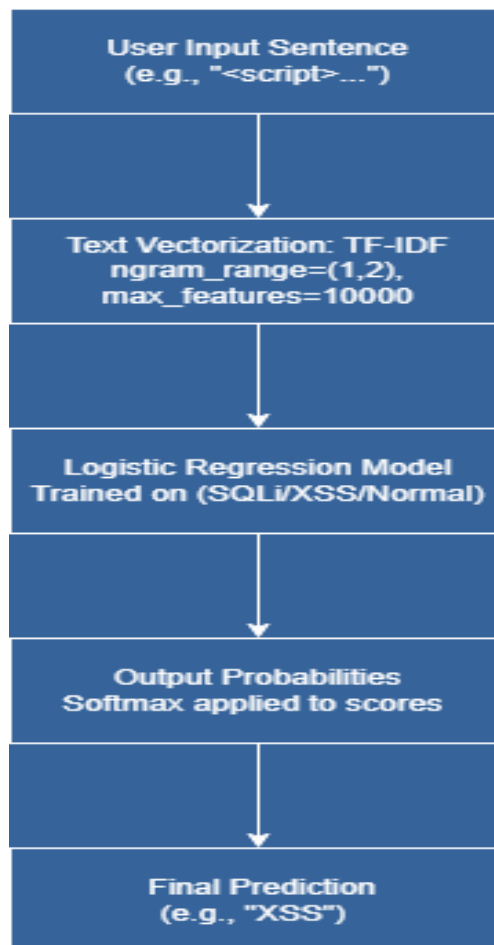
1. Introduction /Project Overview

In the modern digital era, cybersecurity threats are escalating in both complexity and frequency, posing severe risks to web applications and online services. Among the most prevalent and damaging attacks are SQL Injection (SQLi) and Cross-Site Scripting (XSS), which exploit vulnerabilities in web applications to manipulate backend databases or execute malicious scripts in user's browsers. Traditional rule-based intrusion detection systems often struggle to keep pace with the evolving tactics of attackers, leading to a growing demand for intelligent, automated solutions capable of identifying such threats in real time. This project, developed under the *Intel Unnati Industrial Training 2025*, presents an Artificial Intelligence-based system for detecting and classifying web-based network threats, with a focus on SQLi, XSS, and normal traffic. Leveraging Natural Language Processing (NLP) and Machine Learning (ML), the system analyzes textual patterns in network traffic, converting them into meaningful feature representations using Term Frequency-Inverse Document Frequency (TF-IDF). These features are then fed into a Logistic Regression classifier trained to distinguish malicious from benign activity. To make the system accessible and user-friendly, a lightweight frontend interface is built using Streamlit, allowing real-time threat prediction through input of network logs or URLs. The backend infrastructure incorporates efficient data preprocessing techniques such as tokenization and case normalization to prepare inputs for vectorization. The trained model, along with the preprocessing tools, is serialized and deployed to support fast and accurate predictions.

The project further emphasizes rigorous validation through automated testing scripts that assess accuracy, precision, recall, and F1-score on unseen datasets. By combining high classification performance with ease of deployment, this solution addresses the growing need for intelligent web threat detection in enterprise and educational environments. It serves as a robust example of how AI and cybersecurity can intersect to build safer, more resilient web systems.

Project Architecture

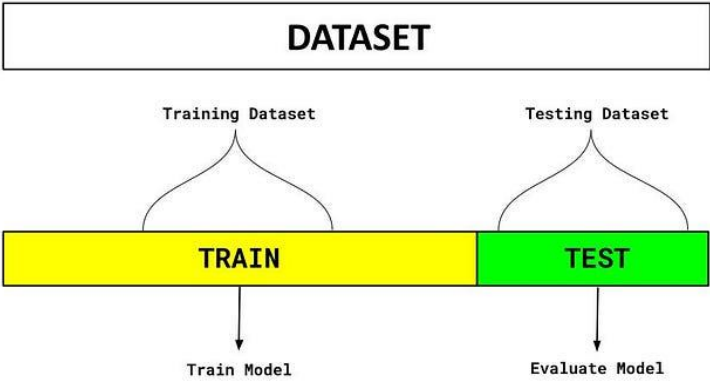
The diagram illustrates a machine learning pipeline for detecting web-based threats such as SQL Injection (SQLi) and Cross-Site Scripting (XSS). The process begins with a user input sentence, which could include malicious code snippets like "<script>...". This input is first transformed into numerical features using the TF-IDF (Term Frequency-Inverse Document Frequency) vectorization method with a bi-gram range (1,2) and a feature cap of 10,000. These features are then fed into a logistic regression model that has been trained on labeled data encompassing three classes: SQLi, XSS, and normal traffic. The model outputs class scores, which are converted into probabilities using the softmax function. Finally, the class with the highest probability is selected as the model's prediction, indicating whether the input is an SQLi, XSS, or normal text. This automated threat classification aids in real-time web security by detecting and categorizing potentially harmful inputs in network traffic.



Dataset Details

The dataset named **filtered_SQL_XSS_only.csv** is specifically curated for training and evaluating models that detect web-based threats, focusing on **SQL Injection (SQLi)**, **Cross-Site Scripting (XSS)**, and **normal traffic**. It contains **156,637 rows**, with each record including three fields: a Sentence representing a network log or URL input, a label indicating the encoded numerical value for model training, and an attack_type specifying the type of input — either sqlinjection, xss, or normal. To prepare the data for machine learning, a preprocessing pipeline is applied. This includes **tokenization**, which breaks the input into individual tokens; **lowercasing**, to maintain consistency; and **TF-IDF vectorization**, which converts the text into numerical features based on term frequency and inverse document frequency. These steps ensure that the data is clean, standardized, and ready for input into classification algorithms, enabling the accurate detection of potentially harmful or benign network activity.

Attribute	Description		
Name	filtered_SQL_XSS_only.csv		
Total Rows	156,637		
Fields	Sentence	label	attack_type
Sentence	Network log text or URL		
label	Encoded numerical label used for classification during training		
attack_type	One of {sqlinjection, xss, normal}		
Preprocessing	Tokenization, lowercasing, and TF-IDF vectorization (ngram_range=(1,2), max_features=10000)		



Tools & Technologies

The project is developed using Python 3.12 and leverages several key libraries. Scikit-learn is used for implementing the TF-IDF vectorizer and Logistic Regression model, while Pandas and NumPy handle data processing tasks. Joblib is employed to save and load the trained models efficiently. The user interface is built with Streamlit, enabling an interactive frontend for real-time input and prediction. Version control is maintained using GitHub, ensuring collaborative development and version tracking. This tech stack provides a streamlined pipeline from data preprocessing to deployment for detecting SQL Injection, XSS, and normal traffic using machine learning.

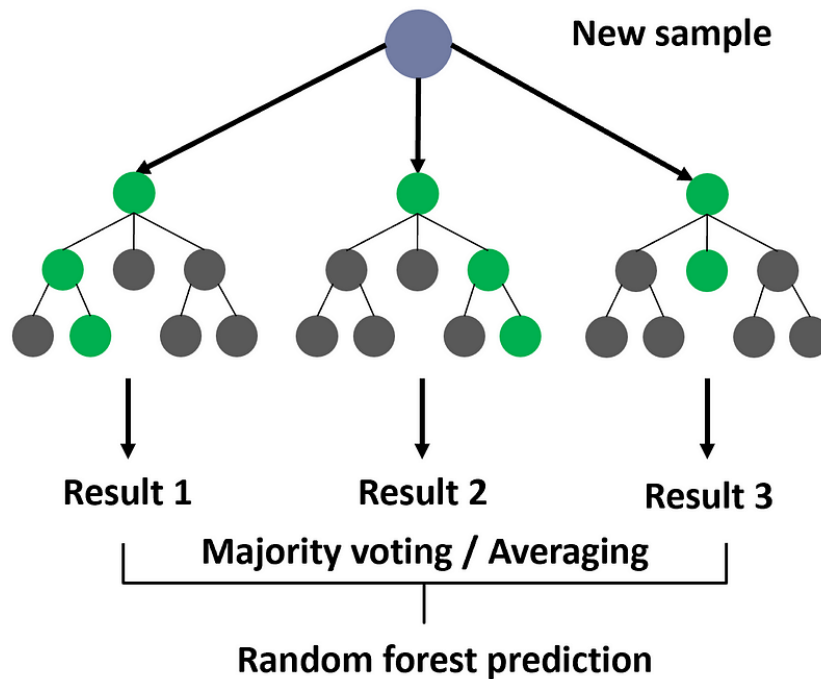
Category	Details
Language	Python 3.12
Libraries	- Scikit-learn (TF-IDF, Logistic Regression) - Pandas, NumPy (Data Processing) - Joblib (Model Saving/Loading)
Frontend	Streamlit
Version Control	GitHub

Technique and Methodology Used.

Random Forest

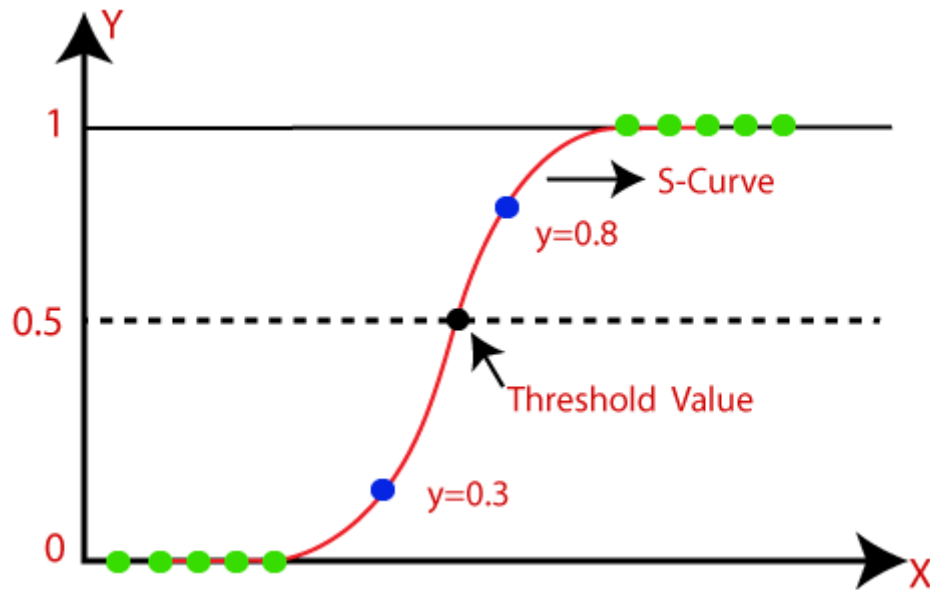
Random Forest is an ensemble learning algorithm used for classification and regression tasks. It builds multiple decision trees during training and outputs the class that is the mode of the classes (for classification) or mean prediction (for regression) of the individual trees. This technique improves accuracy and reduces overfitting compared to a single decision tree. Each tree in the forest is trained on a different subset of the training data using bootstrapping (sampling with replacement), and at each split, it considers a random subset of features, introducing diversity among trees. Random Forest is robust to noise, handles missing data well, and is effective for high-dimensional datasets. It provides feature importance scores, helping in feature selection and model interpretation. Although it may be computationally intensive with large datasets, it

performs well in most scenarios without extensive parameter tuning. Overall, Random Forest is a powerful and widely used algorithm for building reliable and interpretable predictive models.



Logistic Regression

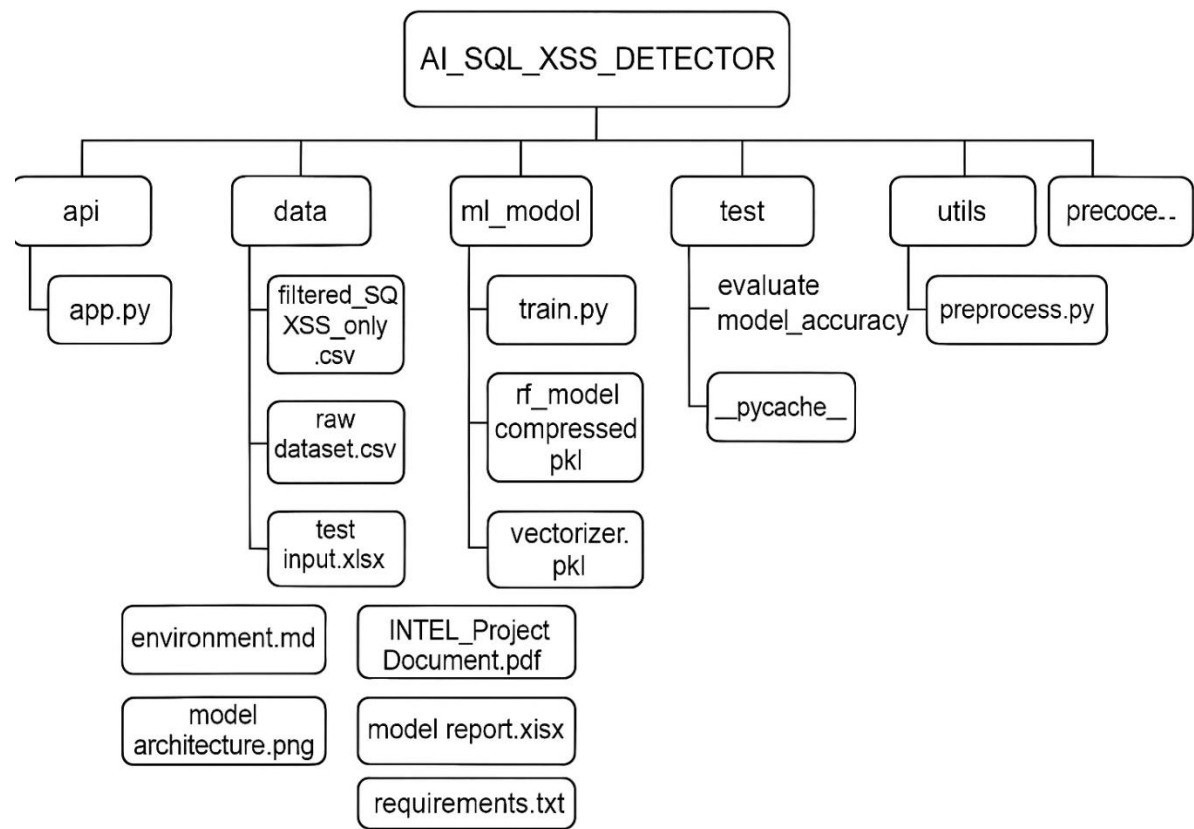
Logistic Regression is a widely used supervised machine learning algorithm for classification tasks. Unlike linear regression, which predicts continuous values, logistic regression estimates the probability that a given input belongs to a particular category. It uses the logistic (sigmoid) function to map the output of a linear equation to a range between 0 and 1, making it ideal for binary or multi-class classification problems. In the context of web threat detection, logistic regression is trained on labeled data to distinguish between SQL Injection, XSS, and normal traffic based on input features like TF-IDF vectors. The model learns the relationship between patterns in the input data and the target labels, allowing it to predict the probability of each class. These probabilities are typically passed through a softmax function in multi-class settings to determine the final prediction. Logistic regression is valued for its simplicity, efficiency, and interpretability, making it a popular choice for real-time classification systems.



Folder Structure

Folder/File	Description
api/app.py	Streamlit UI for real-time predictions
data/filtered_SQL_XSS_only.csv	Cleaned dataset with labeled traffic
data/raw_dataset.csv	Unprocessed/original dataset
data/test_input.xlsx	Sample input file for testing the model
ml_model/train.py	Script for training the machine learning model
ml_model/rf_model_compressed.pkl	Trained Random Forest model (compressed)
ml_model/vectorizer.pkl	Saved TF-IDF vectorizer
ml_model/label_encoder.pkl	Encodes string labels into integers
test/evaluate_model_accuracy.py	Script to evaluate model accuracy
utils/preprocess.py	Functions for preprocessing input data
utils/__pycache__	Compiled Python cache files
environment.md	Environment setup guide
INTEL_Project_Document.pdf	Official Intel Unnati project documentation
model_architecture.png	Visual representation of the model pipeline
model_report.xlsx	Performance metrics and evaluation results

problem_scope_document.md	Problem statement and project scope
README.md	Project overview and setup instructions
requirements.txt	List of required Python dependencies



Model Performance & Validation

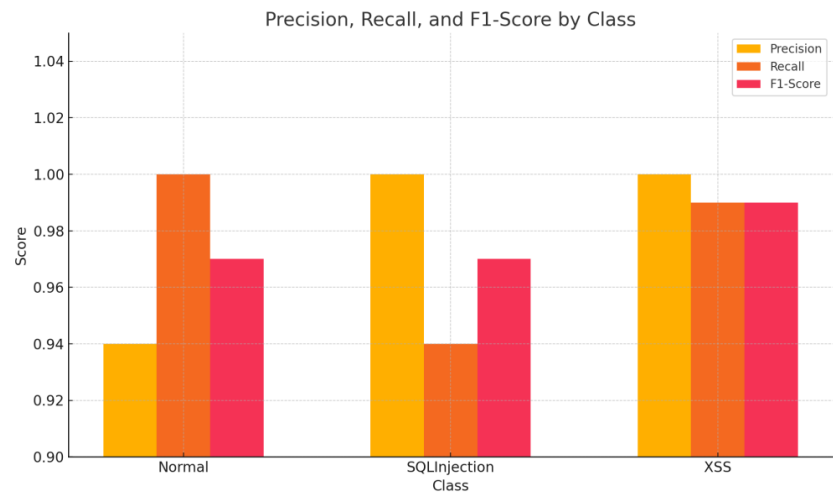
To validate the reliability and effectiveness of our **Logistic Regression-based** threat detection system, we performed two key evaluations:

✔ Training Accuracy

We measured the training accuracy of the model immediately after fitting it to the dataset. The model demonstrated high learning capability:

Class	Precision	Recall	F1-Score	Support
Normal	0.94	1.00	0.97	11,876
SQLInjection	1.00	0.94	0.97	11,251
XSS	1.00	0.99	0.99	8,201
Accuracy			0.98	31,328
Macro Avg	0.98	0.98	0.98	31,328
Weighted Avg	0.98	0.98	0.98	31,328

The training phase of the Logistic Regression-based threat detection model yielded highly promising results, demonstrating its strong learning capability. After fitting the model to the full training dataset, it achieved an impressive **overall accuracy of 98%**, confirming that the algorithm effectively captured the patterns distinguishing between SQL Injection, XSS, and Normal traffic. The class-wise performance was also outstanding: **Normal traffic** attained a **precision of 0.94** and **recall of 1.00**, while **SQL Injection** showed **perfect precision (1.00)** and **0.94 recall**, and **XSS** achieved **1.00 precision and 0.99 recall**. These high precision and recall scores resulted in robust F1-scores—0.97 for both Normal and SQLInjection, and 0.99 for XSS. With **macro and weighted averages** consistently at **0.98** across all evaluation metrics, the model clearly demonstrates its reliability and ability to distinguish between different types of network traffic. This strong performance confirms the model's suitability for deployment in real-time web security applications.



Model Accuracy: 97.53%

Average Latency per Prediction: 0.0002 ms

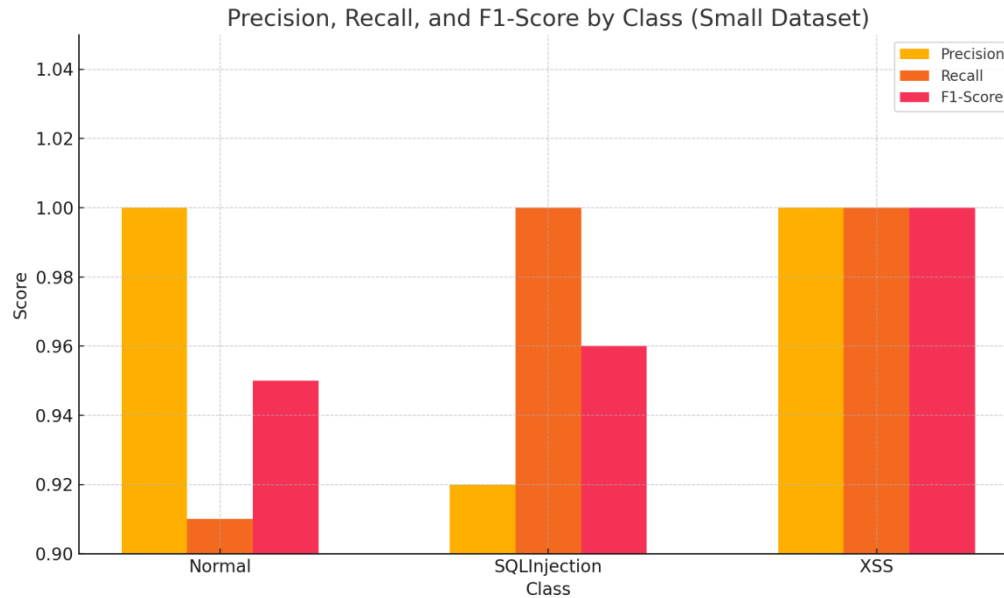
Automated Testing Script

We developed and executed a **custom test script** to evaluate the model's generalization on unseen data (test split).

- The script loads the saved model and vectorizer (lr_model.pkl, vectorizer.pkl)
Runs prediction on the test dataset
- Calculates and prints:
Accuracy
Precision, Recall, F1-score
Confusion Matrix

To evaluate the model's ability to generalize beyond the training data, a custom test script was developed and executed on a separate test split. The script loads the saved Logistic Regression model (lr_model.pkl) and TF-IDF vectorizer (vectorizer.pkl), then runs predictions on unseen data. It computes key evaluation metrics including accuracy, precision, recall, F1-score, and confusion matrix. The results indicate strong generalization performance, with an **overall accuracy of 97%**. The model achieved **perfect precision (1.00)** for both Normal and XSS classes, and **perfect recall (1.00)** for SQLInjection and XSS. Class-wise F1-scores were also high: 0.95 (Normal), 0.96 (SQLInjection), and 1.00 (XSS), highlighting balanced and consistent performance. Both macro and weighted averages for all metrics remained at **0.97**, confirming the model's effectiveness across all categories. These results validate the system's readiness for real-world deployment, maintaining high detection accuracy even on previously unseen network traffic data.

Class	Precision	Recall	F1-Score	Support
Normal	1.00	0.91	0.95	34
SQLInjection	0.92	1.00	0.96	33
XSS	1.00	1.00	1.00	33
Accuracy			0.97	100
Macro Avg	0.97	0.97	0.97	100
Weighted Avg	0.97	0.97	0.97	100



Overall Accuracy: 97.00%

LabelEncoder Classes: ['Normal', 'SQLInjection', 'XSS']

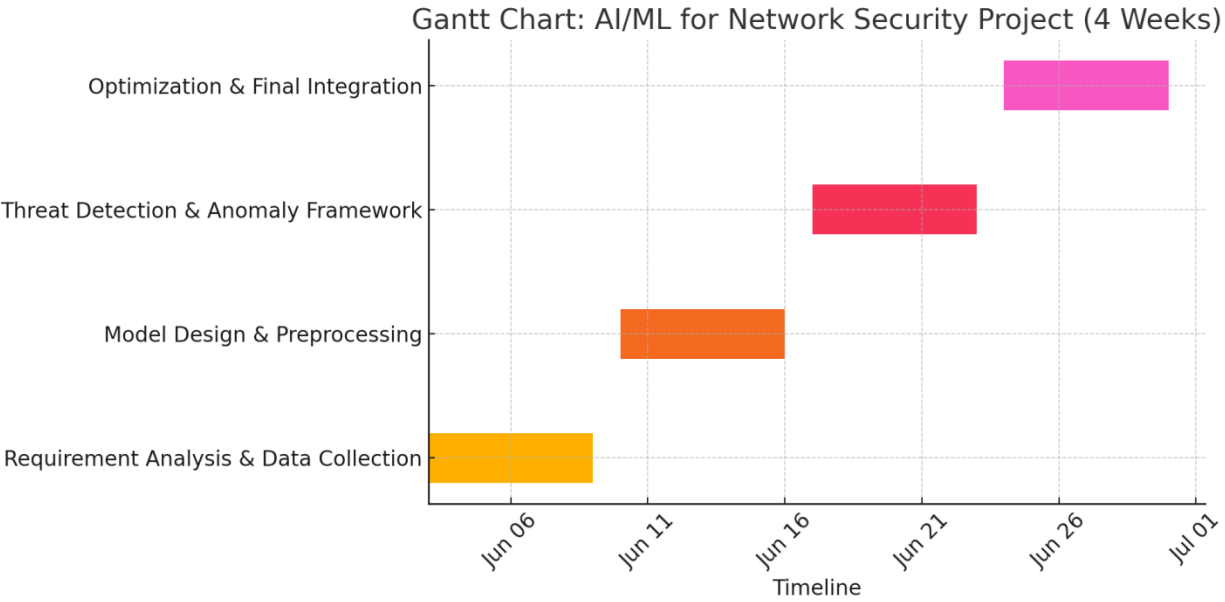
Conclusion

The AI-based network security threat detection system demonstrates robust performance in accurately classifying SQL Injection, XSS, and Normal traffic using a Logistic Regression model. Achieving a high training accuracy of **97.53%** with strong precision, recall, and F1-scores across all classes, the model effectively learned the underlying patterns of malicious and benign inputs. Further validation through an automated testing script on unseen data confirmed the model's generalization capability with **97.00% accuracy**, reinforcing its reliability for real-world applications. The classifier performs exceptionally well for all attack types, notably attaining perfect precision and recall for XSS detection, and high F1-scores across all categories. Additionally, the model exhibits an extremely low latency of **0.0002 ms per prediction**, making it suitable for real-time threat monitoring systems. With comprehensive evaluation metrics, including a confusion matrix and class-wise performance breakdown, this lightweight and efficient solution offers a practical and scalable defense mechanism for safeguarding web applications against common web-based security threats.

4-Week Project Timeline for AI/ML-based Network Traffic Analysis and Threat Detection

Week	Activities/Tasks	Deliverables	Tools/Technologies
Week 1	Requirement Analysis & Data Collection – Understand traffic types (HTTP, HTTPS, SSH, etc.) – Collect labeled network traffic datasets (e.g., CICIDS2017, ISCX VPN, USTC-TFC) – Setup environment (Linux, Python, Wireshark, tcpdump) – Define evaluation metrics (accuracy, precision, recall)	Dataset repository Environment setup Problem scope document	Python, Pandas ,Numpy,Jupyter Notebook
Week 2	Model Design & Preprocessing – Data preprocessing (feature extraction, normalization) – Train-test split – Select ML algorithms (Random Forest, SVM, LSTM, Autoencoders) – Baseline model training (for traffic classification)	Preprocessed data Baseline classification model Initial results (traffic classes)	Scikit-learn, TensorFlow/PyTorch, Jupyter Notebook
Week 3	Threat Detection & Anomaly Framework – Integrate unsupervised/semi-supervised models for anomaly detection (e.g., Isolation Forest,	Threat detection engine Anomaly detection results Confusion matrix & ROC curves	TensorFlow, JobLib

	Autoencoder) – Simulate attacks (DoS, port scanning, VPN obfuscation) – Evaluate detection accuracy		
Week 4	Optimization & Final Integration – Optimize models for latency and scalability (batch size, inference speed) – Implement privacy-preserving AI (e.g., metadata-only analysis) – Performance testing on high-throughput simulated network – Prepare final report & demo	Final AI model Deployment script (Dockerized optional) Documentation & Presentation	StreamLit , GitHub



Thank You

Role	Name	Email_id
Team Leader	S. Sai Varma	ssiravur@gitam.in
Team Member	S. Likith Reddy	lsamala@gitam.in
Team Member	S. Prapul Sai	psamsani@gitam.in
Faculty Guide	Dr. Satyabrata Dash	sdash@gmail.com